

Dynamic Programming

INTRODUCCIÓN TEÓRICO-PRÁCTICA AL APRENDIZAJE POR REFUERZO

Profesor: Matias Bilkis

Cuatrimestre: Segundo cuatrimestre 2026

Integrantes

Agustín Calo	390/23	Paz Lavenia	64/23
Rocco Gasparini	102/24	Silvano Picard	637/24
Maria Delfina Kiss	72/23	Ingrid von Foerster	150/23

Análisis de Value Iteration

Usamos el código base de *Value Iteration* que se dio en clase. El entorno modelado es un laberinto representado como una grilla de $n \times n$ con un 20 % de paredes ubicadas al azar y un estado objetivo. El objetivo del experimento fue variar el tamaño de la grilla (desde $n = 10$ hasta $n = 1000$, en saltos de 100 a partir de $n = 1000$) para evaluar cómo escala el algoritmo.

A partir de las ejecuciones, generamos los gráficos de rendimiento (ver Figura 1) y notamos algunos comportamientos:

Análisis de Escalabilidad: Value Iteration en Laberintos Aleatorios

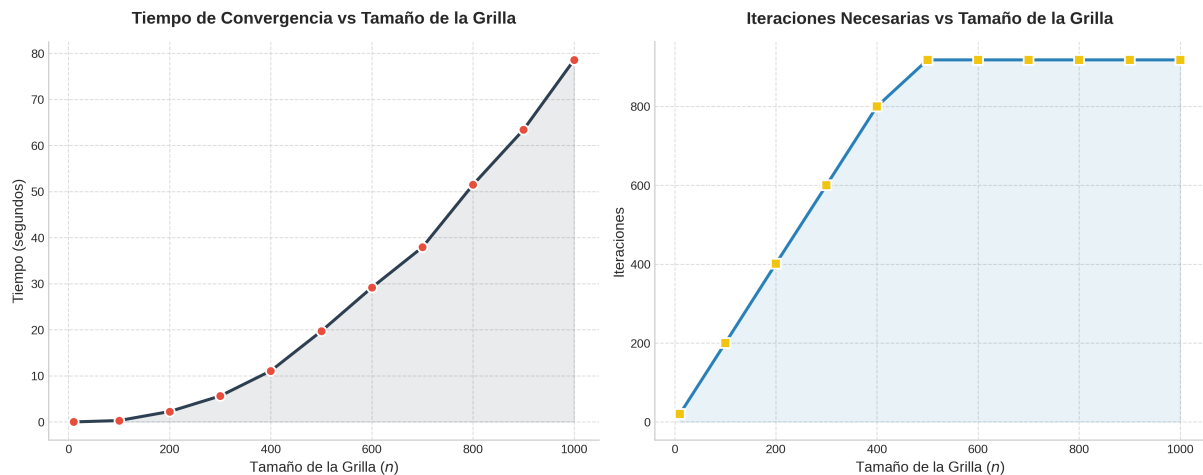


Figura 1: Tiempo de convergencia e iteraciones necesarias en función del tamaño de la grilla.

- **Meseta en las iteraciones (gráfico derecho):** Al principio, la cantidad de barridos crece linealmente. Sin embargo, a partir de $n = 500$, la curva se estanca por completo en torno a las 900 iteraciones. Esto ocurre por la combinación del factor de descuento ($\gamma = 0.99$) y el umbral de tolerancia ($\theta = 10^{-4}$). Al ser la grilla tan masiva, la recompensa se atenúa tanto con la distancia que, para los estados más alejados, las actualizaciones caen por debajo de θ y el algoritmo corta antes de que la señal se propague por todo el laberinto.
- **Crecimiento polinómico del tiempo (gráfico izquierdo):** La curva temporal se dispara, alcanzando casi los 80 segundos para la grilla más grande. Esto refleja el costo de evaluar matrices donde el total de estados crece cuadráticamente (n^2). Para $n = 1000$, estamos operando sobre un millón de celdas por iteración.

En conclusión, llevar la grilla hasta $n = 1000$ ilustra el límite práctico de *Value Iteration*. No

solo el tiempo de procesamiento se dispara por el volumen del espacio de estados, sino que la atenuación geométrica de las recompensas impide que el algoritmo resuelva los caminos óptimos completos en distancias extremadamente largas bajo estos hiperparámetros.

Capítulo 4: Programación Dinámica

Definimos la **Programación Dinámica (DP)** como un conjunto de algoritmos diseñados para calcular políticas óptimas asumiendo un modelo perfecto del entorno modelado como un MDP. Aunque son de utilidad práctica limitada por su alto costo computacional y la necesidad de un modelo perfecto, son teóricamente fundamentales; el resto de los métodos de RL buscan imitar su efecto con menos cómputo y sin modelos exactos.

Se trabaja bajo la premisa de que los conjuntos de estados (S), acciones ($A(s)$) y recompensas (R) son finitos, rigiéndose por probabilidades de transición $p(s', r|s, a)$.

Los algoritmos de programación dinámica se construyen tomando las ecuaciones de optimalidad de Bellman y transformándolas en reglas de actualización iterativas (asignaciones) para mejorar aproximaciones sucesivas de las funciones de valor.

Recuerdo las ecuaciones de Bellman para $v^*(s)$ y $q^*(s, a)$:

$$v^*(s) = \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma v^*(s')]$$

$$q^*(s, a) = \sum_{s', r} p(s', r|s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]$$

Evaluación de Políticas

La **Evaluación de Políticas** constituye el proceso iterativo destinado a calcular la función de valor v_π bajo una política arbitraria π . Estas operaciones se ejecutan mediante **Full Backups**, las cuales consideran todas las transiciones posibles de un solo paso junto con sus respectivas recompensas esperadas, prescindiendo del uso de muestras individuales. Para optimizar el rendimiento computacional, se implementa la **actualización in-place**, empleando una única estructura de datos donde los nuevos valores sobrescriben de inmediato a los anteriores durante cada barrido del espacio de estados, acelerando así la convergencia del algoritmo. Dicho proceso iterativo se detiene al satisfacer la **condición de parada**, la cual evalúa que el cambio máximo entre estimaciones sucesivas sea inferior a un umbral θ :

$$\max_{s \in S} |v_{k+1}(s) - v_k(s)| < \theta$$

Policy Improvement

Para saber si conviene cambiar de acción en un estado s , evaluamos el valor de tomar dicha acción y luego seguir la política actual π , formalizado mediante $q_\pi(s, a)$. el Teorema de **Policy Improvement** establece que si una nueva política π' cumple con que $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ para todo estado, entonces π' es tan buena o mejor que π ($v_{\pi'}(s) \geq v_\pi(s)$). Esto es una política greedy pues la nueva política se obtiene seleccionando en cada estado la acción que maximiza la función de valor actual: $\pi'(s) = \arg \max_a q_\pi(s, a)$.

Policy Iteration

Mediante la alternancia sucesiva entre la evaluación de políticas ($\xrightarrow{E}$) y su respectiva mejora ($\xrightarrow{I}$), se genera una secuencia monótona que converge hacia la política y función de valor óptimas:

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi^* \xrightarrow{E} v^*$$

Dado que cada paso garantiza una mejora estricta (salvo que se alcance la optimalidad) y que un MDP finito posee un número acotado de políticas, este procedimiento iterativo, conocido como **Policy Iteration**, converge de manera exacta en una cantidad finita de pasos.

Value Iteration

La **Value Iteration** es un algoritmo que simplifica la evaluación de políticas al limitarla a un único *backup* por estado. Al incorporar el operador de máximo ($\max_a$) de manera directa sobre la ecuación de optimalidad, fusiona la evaluación truncada y la mejora de políticas en una única regla de actualización:

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

A partir de cualquier función de valor inicial arbitraria v_0 , este procedimiento iterativo garantiza la convergencia matemática hacia la función de valor óptima (v_*).

Generalized Policy Iteration (GPI)

La iteración de políticas consta de dos procesos simultáneos e interactivos, uno que hace que la función de valor sea consistente con la política actual (policy evaluation) y el otro que hace que la política sea golosa con respecto a la función de valor actual (policy improvement). Utilizamos el término **generalized policy iteration** para referirnos a la idea general de permitir que los procesos de evaluación de políticas y mejora de políticas interactúen, independientemente de la granularidad y otros detalles de los dos procesos.

Ambos procesos se estabilizan simultáneamente solo cuando se alcanza la optimalidad (cuando la función de valor es consistente con la política y esta es golosa respecto a dicha función, satisfaciendo la ecuación de Bellman).